

Fuzzy Hashing

RHOULAM Fatima Zahra , SININI Emmanuel

1 INTRODUCTION

1.1 CONTEXTE ET PROBLÉMATIQUE

Face à l'explosion massive des données numériques, les **fonctions de hachage cryptographiques** classiques (telles que SHA-256 ou MD5) montrent leurs limites structurelles. Leur propriété fondamentale, l'effet d'avalanche, rend impossible la détection de fichiers modifiés ou de contenus similaires : la modification d'un seul bit produit une empreinte radicalement différente. L'enjeu actuel réside donc dans la capacité à identifier des **variantes** (malwares polymorphes, quasi-doublons, médias compressés) plutôt que des copies binaires parfaites.

1.2 LES DEUX PILIERS DE LA SIMILARITÉ

Pour répondre à ce besoin d'identification flexible, deux approches complémentaires se distinguent :

- **Fuzzy Hashing (Niveau bit)** : Analyse la structure binaire brute pour détecter des fragments communs ou des séquences d'octets partagées (ex. : *ssdeep*, *TLSH*).
- **Perceptual Hashing (Niveau sémantique)** : Analyse le contenu tel qu'il est perçu par l'humain. Il ignore les changements de format ou de compression pour se concentrer sur les caractéristiques visuelles ou auditives (ex. : *pHash*, *PDQ*).

1.3 OBJECTIFS DU RAPPORT

Ce travail présente l'état de l'art de ces technologies. Il analyse leur **robustesse** (résistance aux modifications volontaires ou accidentelles), leur **pouvoir discriminant** (capacité à limiter les faux positifs) et compare les performances des algorithmes leaders. L'objectif final est de définir les critères de sélection optimaux selon l'usage métier : *digital forensics*, modération de contenu ou protection du *copyright*.

2 CONCEPTS DE BASE

2.1 DÉFINITIONS ET TYPOLOGIE

Il est crucial de distinguer trois familles de fonctions de hachage selon leurs objectifs :

- **Hash Cryptographique** : Projection d'une entrée de taille arbitraire vers une chaîne fixe. Une modification de l'entrée $x \neq x'$ implique $H(x) \neq H(x')$ avec une probabilité quasi-certaine (résistance aux collisions).
- **Fuzzy Hashing / Similarity Digest** : Conçu pour détecter des similarités au niveau binaire. Si deux fichiers A et B sont proches, leurs empreintes $F(A)$ et $F(B)$ le seront aussi.
- **Perceptual Hashing (pHash)** : Extraction de caractéristiques sémantiques. Le hash reste stable tant que la perception humaine du média (image, son) ne change pas, malgré des modifications de signal (bruit, redimensionnement).

2.2 PROPRIÉTÉS ET MÉTRIQUES DE PERFORMANCE

L'efficacité d'un algorithme de hachage de similarité s'évalue selon quatre propriétés fondamentales :

1. **Robustesse** : Capacité à générer des hashes proches malgré des modifications (compression, édition).
2. **Pouvoir discriminant** : Capacité à différencier deux contenus distincts (éviter les faux positifs).
3. **Compacité** : Taille du digest par rapport au fichier original.
4. **Efficacité** : Vitesse de calcul et de comparaison des empreintes.

2.3 MESURE DE LA SIMILARITÉ

Contrairement au hachage classique, la comparaison repose sur une fonction de distance D . La similarité est confirmée si :

$$D(H_1, H_2) \leq \tau \quad (1)$$

Où τ représente le seuil de tolérance (threshold). Les distances les plus communes sont :

- **Distance de Hamming** : Nombre de bits divergents (utilisée en hachage perceptuel).
- **Distance d'édition (Levenshtein)** : Nombre d'opérations pour transformer une chaîne en une autre (utilisée par *ssdeep*).
- **Distance TLSH** : Score spécifique basé sur la distribution des quartiles.

2.4 DESCRIPTION DES ALGORITHMES DE HACHAGE DE FICHIERS

ssdeep (Context Triggered Piecewise Hashing) L'algorithme *ssdeep* repose sur le concept de découpage par contenu.

- **Segmentation dynamique** : Au lieu de découper le fichier en blocs de taille fixe, il utilise un *rolling hash* (Adler-32) qui définit une limite de bloc dès qu'une signature spécifique est rencontrée. Cela permet de garder les mêmes blocs même si des données sont insérées au début du fichier.
- **Signature** : Pour chaque bloc, un hash simple (FNV) de 6 bits est calculé.
- **Double hachage** : Le digest contient deux signatures (pour une taille de bloc n et $2n$) afin de permettre la comparaison de fichiers de tailles différentes.

sdhash (Similarity Digests) *sdhash* est une approche basée sur la sélection de caractéristiques (*features*).

- **Sélection par entropie** : L'algorithme parcourt le fichier avec une fenêtre glissante de 64 octets et ne conserve que les séquences ayant une entropie inhabituelle (les plus discriminantes).
- **Filtres de Bloom** : Les séquences choisies sont hachées (SHA-1) et insérées dans une série de filtres de Bloom de 256 octets.
- **Comparaison** : La similarité est mesurée par le taux d'intersections entre les filtres de Bloom de deux fichiers.

TLSH (Trend Micro Locality Sensitive Hash) TLSH est une approche statistique robuste conçue pour le clustering à grande échelle.

- **Glissement de fenêtre** : Il utilise une fenêtre de 5 octets pour compter les occurrences de triplets d'octets (n-grams).
- **Construction de l'histogramme** : Ces comptes remplissent un tableau de 128 seaux (buckets).
- **Calcul des quartiles** : Le hash final code la position des valeurs de l'histogramme par rapport aux quartiles ($Q1, Q2, Q3$).
- **Distance** : Le score TLSH (distance) est calculé par la différence entre les quartiles et les valeurs des seaux. Un score de 0 indique une identité parfaite.

2.5 COMPARAISON SYNTHÉTIQUE DES APPROCHES FICHIERS

Algorithme	Force principale	Faiblesse majeure
ssdeep	Très rapide, standard industriel.	Très sensible au réordonnement des blocs.
sdhash	Capable de trouver un fichier imbriqué dans un autre.	Lent et génère des digests volumineux.
TLSH	Très robuste aux attaques adversariales (bruit).	Nécessite un fichier d'au moins 50 octets.

Table 1: Comparatif ssdeep vs sdhash vs TLSH

2.6 ANALYSE DES TECHNIQUES DE HACHAGE D'IMAGES

Average Hash (aHash) : L'approche par seuillage de luminance L'algorithme *aHash* est le plus rudimentaire. Il repose sur la structure globale de l'image :

1. Réduction de l'image à une taille de 8×8 pixels.
2. Conversion en niveaux de gris et calcul de la moyenne de luminance μ .
3. Pour chaque pixel P_i , le bit du hash $b_i = 1$ si $P_i > \mu$, sinon 0.

Analyse : Bien qu'ultra-rapide, *aHash* est vulnérable aux modifications de gamma ou aux filtres qui altèrent la moyenne globale des pixels.

Perceptual Hash (pHash) : L'analyse fréquentielle (DCT) Le *pHash* est beaucoup plus fin car il analyse les fréquences visuelles plutôt que les valeurs de pixels brutes.

- **Transformation** : On applique une DCT (Discrete Cosine Transform) sur une image 32×32 .
- **Filtrage** : On ne conserve que le bloc 8×8 en haut à gauche (basses fréquences). Ce bloc représente les formes structurelles de l'image sans les détails (bruit).
- **Quantification** : On compare chaque coefficient à la médiane des fréquences.

Analyse : C'est la méthode la plus robuste face aux attaques courantes (compression JPEG forte, flou, légères rotations).

Difference Hash (dHash) : L'approche par gradient Le *dHash* se concentre sur la structure des bords et des contrastes.

- Il compare chaque pixel au pixel adjacent (gauche vers droite).
- Si $P_x > P_{x+1}$, le bit est à 1, sinon 0.

Analyse : Contrairement à *aHash*, il peut résister à un changement de luminosité global (car la *différence* entre deux pixels reste identique).

Wavelet Hash (wHash) : L'analyse multi-résolution Basé sur la **Discrete Wavelet Transform** (DWT), cet algorithme est similaire au pHash mais remplace la DCT par des ondelettes (souvent de Haar). *Avantage* : Il capture mieux la localisation spatiale des caractéristiques, ce qui le rend très efficace pour les images texturées.

2.7 TABLEAU COMPARATIF DE LA ROBUSTESSE

Attaque / Transformation	aHash	pHash	dHash
Redimensionnement	Élevée	Excellente	Excellente
Compression JPEG	Moyenne	Excellente	Élevée
Modification de luminosité	Faible	Élevée	Excellente
Rotation légère ($< 5^\circ$)	Faible	Moyenne	Faible
Filigrane (Watermark)	Faible	Élevée	Moyenne

Table 2: Comparaison de la robustesse des algorithmes de hachage d'images

2.8 PERCEPTUAL HASHING VIDÉO ET AUDIO

Hachage Vidéo : L'approche spatio-temporelle Le hachage vidéo ne peut se résumer à une simple suite de hashes d'images, car cela générerait des signatures trop volumineuses et sensibles au décalage temporel. Deux algorithmes de référence (développés par Meta/Facebook) dominent l'état de l'art :

- **PDQ** : Génère un hash de 256 bits par frame clé. Il utilise une transformation en cosinus discrète (DCT) modifiée pour être plus robuste aux rotations et aux incrustations de texte.
- **TMK+PDQF (Temporal Match Kernel)** : Cet algorithme agrège les hashes PDQ des frames sur toute la durée du clip. Il crée une signature unique capable d'identifier une séquence même si elle est accélérée, ralentie ou si des segments ont été supprimés.

Hachage Audio : Empreintes acoustiques Le hachage audio (Acoustic Fingerprinting) repose sur la transformation du signal sonore en un **spectrogramme** (représentation temps-fréquence).

- **Extraction de points d'intérêt** : On identifie les pics d'énergie dans le spectrogramme (les "constellations").
- **Robustesse** : Cette méthode permet d'identifier un morceau même en présence de bruit ambiant, d'une compression MP3 agressive ou d'une modification de l'égalisation.
- **Deep Audio Hashing** : Les approches récentes comme *AudioNet* utilisent des réseaux de neurones pour créer des *embeddings* (hashs appris) encore plus résistants aux distorsions complexes.

2.9 PERFORMANCE ET TOLÉRANCE AUX DIFFÉRENCES

En multimédia, la performance est évaluée par le compromis entre :

- 1. **Robustesse** : Le hash doit rester inchangé face aux transformations dites “bénignes” (redimensionnement, changement de bitrate).
- 2. **Fragilité aux modifications malveillantes** : Le hash doit changer radicalement si le contenu est détourné (ex: Deepfake, modification du sens d’une scène).
- 3. **Taille du digest** : Pour la modération à l’échelle d’Internet, les hashes doivent être compacts (ex: 256 bits) pour permettre des milliards de comparaisons par seconde.

2.10 ALGORITHMES DE RÉFÉRENCE POUR L’AUDIO ET LA VIDÉO

Algorithmes Audio (Fingerprinting) L’audio nécessite de capturer des caractéristiques qui survivent au bruit ou à la distorsion.

- **Shazam (Algorithm de Wang)** : Le plus célèbre. Il crée une carte de points d’intérêt (Landmarks) basée sur les pics d’énergie dans le spectrogramme. Il lie ces points par paires pour créer des hashes résistants au bruit ambiant.
- **Chromaprint (AcoustID)** : Utilise l’analyse des "Chroma features" (notes de musique) pour identifier des morceaux de musique. C’est l’algorithme derrière les bases de données open-source.
- **Audio Fingerprinting de Philips** : Basé sur la différence d’énergie entre des bandes de fréquences consécutives.

Algorithmes Vidéo (Spatiotemporel) La vidéo ajoute la contrainte de la durée et du mouvement.

- **PDQ (Meta)** : Un algorithme de hachage d’image optimisé pour être transformé en hachage vidéo. Il est utilisé pour détecter les contenus interdits sur Facebook et Instagram.
- **TMK (Temporal Match Kernel)** : Souvent couplé à PDQ. Il ne regarde pas seulement les images, mais *comment* elles s’enchaînent. Il peut identifier une vidéo même si on y a inséré des publicités ou si on l’a remontée.
- **VGG / ResNet (Deep Hashing)** : Aujourd’hui, on utilise des réseaux de neurones (CNN) pour extraire des vecteurs de caractéristiques (embeddings). On compare ces vecteurs via la distance cosinus.
- **Video-Multimethod Assessment Fusion (VMAF)** : Bien que principalement pour la qualité, certains dérivés sont utilisés pour comparer la structure temporelle des flux.

Nom	Domaine	Usage principal
Shazam	Audio	Reconnaissance de musique en milieu bruyant.
AcoustID	Audio	Nettoyage de bibliothèques musicales (Metadata).
PDQ / TMK	Vidéo	Modération de contenu et lutte contre le piratage.
Fingerprinting NIST	Vidéo	Utilisé dans les enquêtes judiciaires (Forensics).

Table 3: Algorithmes célèbres en hachage multimédia

Comparaison des objectifs

3 COMPARAISON TRANSVERSALE ET SYNTHÈSE

3.1 TABLEAU COMPARATIF GLOBAL

Le tableau suivant résume les différences de paradigmes entre le hachage de similarité binaire et le hachage perceptuel multimédia.

Caractéristique	Fuzzy Hashing (Fichiers)	Perceptual Hashing (Média)
Niveau d'analyse	Octets / Structure binaire brute.	Caractéristiques sémantiques / Fréquences.
Entrée type	Exécutables, documents, archives.	Images, flux audio, fichiers vidéo.
Sensibilité au format	Très élevée (un PDF vs un Docx du même texte sont différents).	Faible (indifférent à la compression ou au conteneur).
Métrique principale	Distance d'édition ou distance TLSH.	Distance de Hamming ou distance Cosinus.
Cas d'usage	Malware forensics, détection de fragments de code.	Copyright, modération, recherche par l'image.

Table 4: Synthèse comparative entre approches binaires et perceptuelles

3.2 ANALYSE DE LA TOLÉRANCE À LA DIFFÉRENCE

La notion de "proximité" varie radicalement selon le domaine d'application :

En Digital Forensics (ssdeep/sdhash) La tolérance est souvent faible. On cherche à prouver que deux fichiers partagent un ancêtre commun ou des fragments de code identiques. Un score de similarité de 30% peut déjà être considéré comme une preuve de parenté technique (ex: variantes d'un même ransomware).

En Modération de contenu (pHash/PDQ/TMK) La tolérance est beaucoup plus large. Le but est de reconnaître une œuvre même après des transformations agressives (recadrage de 20%, ajout de bruit, changement de pitch audio). Ici, on accepte une dérive plus importante du hash tant que la "perception" reste la même pour l'humain.

3.3 LE COMPROMIS SÉCURITÉ VS PERFORMANCE

- **Collisions intentionnelles** : Les hashes perceptuels et fuzzy sont, par définition, plus vulnérables aux collisions que les hashes cryptographiques. Un attaquant peut tenter de modifier un fichier juste assez pour que le hash change, mais pas assez pour que l'humain perçoive la différence.
- **Scalabilité** : Plus l'algorithme est précis (comme *sdhash*), plus il est coûteux en calcul. À l'inverse, *TLSH* et *pHash* sont optimisés pour des comparaisons à l'échelle de bases de données de plusieurs millions d'entrées.

3.4 SYNTHÈSE : QUAND CHOISIR QUEL ALGORITHME ?

1. **Besoin d'identifier des versions de documents texte/code** : Privilégier *ssdeep*.
2. **Clustering massif de malwares** : Privilégier *TLSH*.
3. **Recherche de contenus multimédia identiques (déduplication)** : Privilégier *dHash*.
4. **Protection de droits d'auteur (robuste aux attaques)** : Privilégier *pHash* ou *TMK+PDQ*.